

NYCU-EE IC LAB – Fall 2025

Lab07 Exercise

Design: Number Theoretic Transform (NTT)

Data Preparation

1. Extract test data from TA's directory:
`% tar -xvf ~iclabTA01/Lab07.tar`
2. The extracted LAB directory contains:
 - a. **00_TESTBED**
 - b. **01_RTL**
 - c. **02_SYN**
 - d. **03_GATE**
 - e. **04_MEM**
 - f. **05_JG**

Basic Concept

Number Theoretic Transform (NTT) are used in cryptography. Like the Fast Fourier Transform, NTT helps reduce the calculation steps for polynomial multiplication. The difference is that NTT operates over a finite field, specifically a polynomial ring.

In this lab, you should calculate NTT of a given polynomial with degree n is 128. The detailed structure is described below.

1. The input polynomial is given in clk1 domain.
2. Use Handshake synchronizer to transfer the polynomial into clk2 domain.
3. Calculate the NTT in clk2 domain.
4. Store the result into FIFO synchronizer.
5. Get the result from FIFO synchronizer and output the result in the clk3 domain.

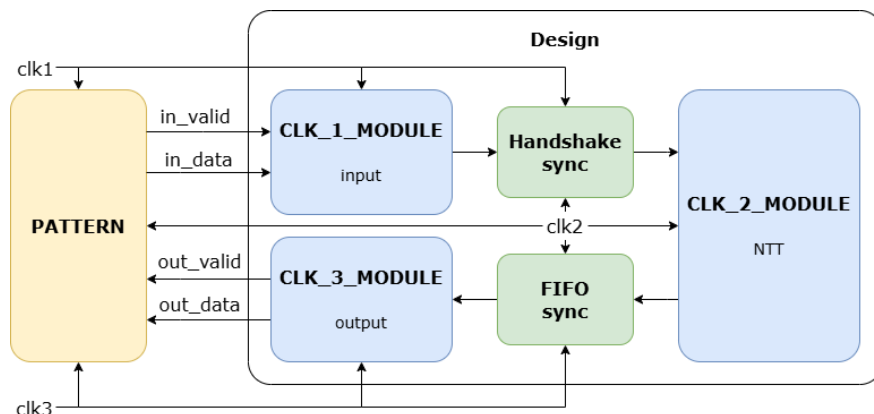


Fig1. Block diagram

Design Description

In this lab, you are asked to implement an NTT with a specified degree of 128 and a given modulus **Q**. The algorithm is described below:

NTT (x)	
1	t = 128
2	for (m = 1; m < 128; m = m * 2)
3	ht = t / 2
4	for (i = 0, j1 = 0; i < m; i = i + 1, j1 = j1 + t)
5	s = GMb [m + i]
6	j2 = j1 + ht
7	for (j = j1; j < j2; j = j + 1)
8	u = x [j]
9	v = modq_mul (x [j + ht], s)
10	x [j] = (u + v) mod Q
11	x [j + ht] = (u - v) mod Q
12	t = ht;

modq_mul (a, b)	
// Montgomery multiplication	
1	R = 2 ¹⁶
2	x = a * b
3	y = (x * Q0I) mod R
4	z = (x + y * Q) / R
5	if (z >= Q) return z - Q
6	else return z

The constant **Q** is 12289, **Q0I** is 12287, and the twiddle factor **GMb** are given in **GMb.txt** within 00_TESTBED.

Here're some notice and hint. First, $(a - b) \bmod Q$ should follow:

$$(a - b) \bmod Q = \begin{cases} a - b, & a \geq b \\ a - b + Q, & a < b \end{cases}$$

Second, an example structure diagram with degree 8 is provided in following figures. Figure 2 shows the full picture of how the algorithm processes the coefficients with several butterfly units, and Figure 3 establishes the details of a single butterfly unit.

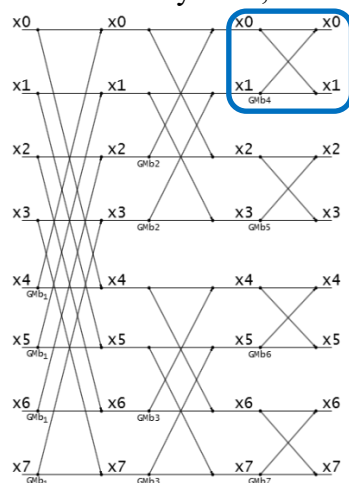


Fig2. Example structure diagram

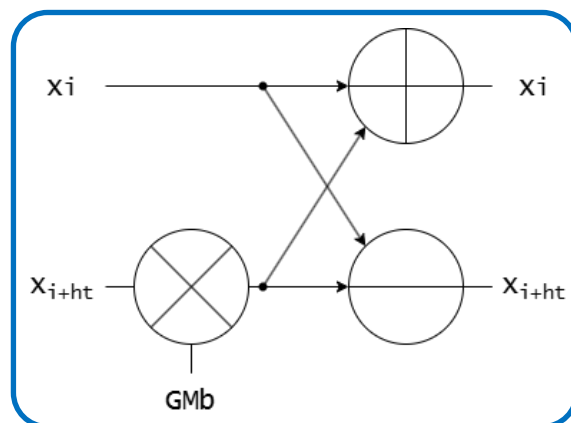


Fig3. Multiplier & butterfly unit

The corresponding hardware architecture, a *pipelined radix-2 NTT*, is shown in Fig.4, each stage includes a multiplier, a butterfly, a buffer, and some mux. One can start from such structure and get improvement through the input/output behavior.

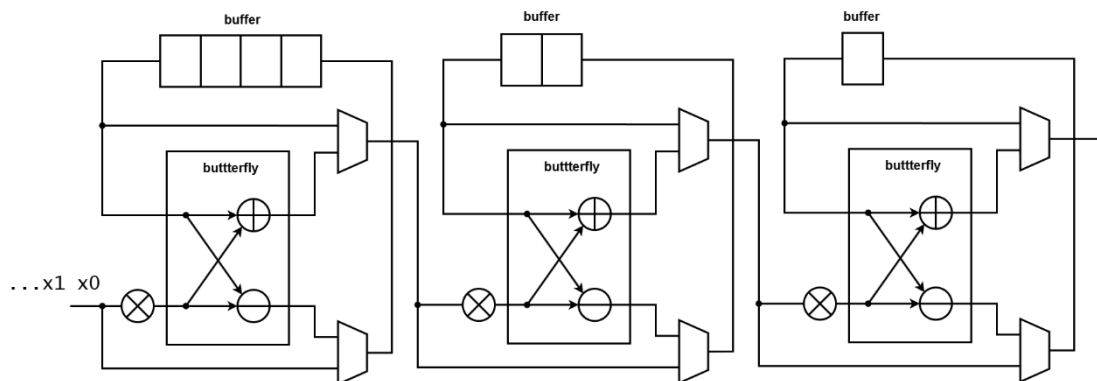


Fig4. A common pipelined 8-point radix-2 FFT architecture

In this lab, you will also deal with the CDC (clock domain crossing) problem. The Handshake synchronizer is used to cross clk1 to clk2. The FIFO synchronizer is used to cross clk2 to clk3. The circuit structures are shown below:

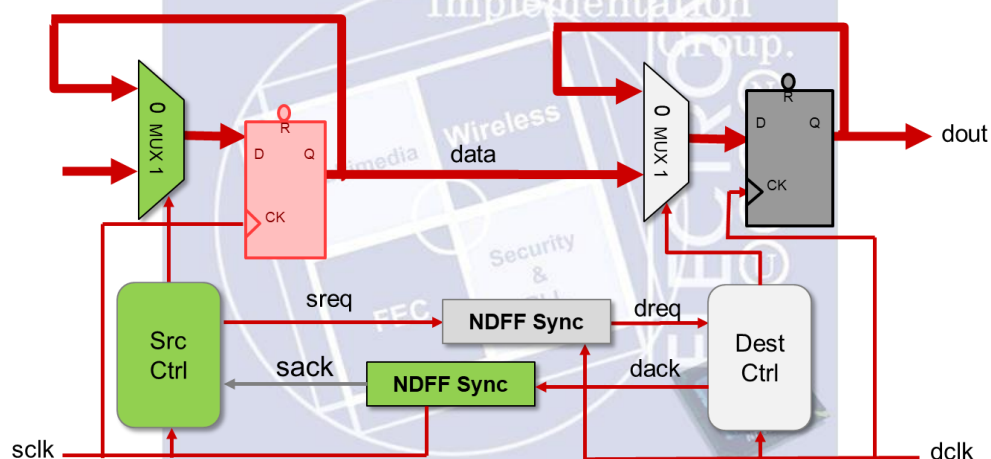


Fig5. Handshake synchronizer circuit structure

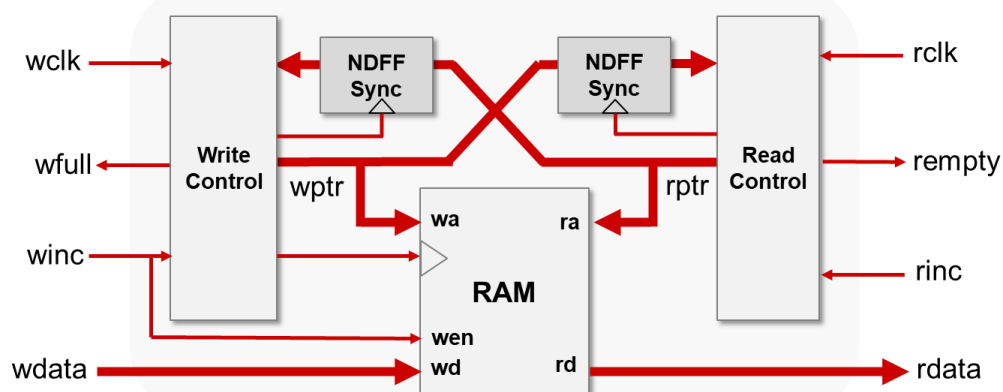


Fig6. FIFO synchronizer circuit structure

In this lab, you should use JG to verify the CDC design. After run the JG, there should not be no error message in console, no violation message and nothing in violation.csv.

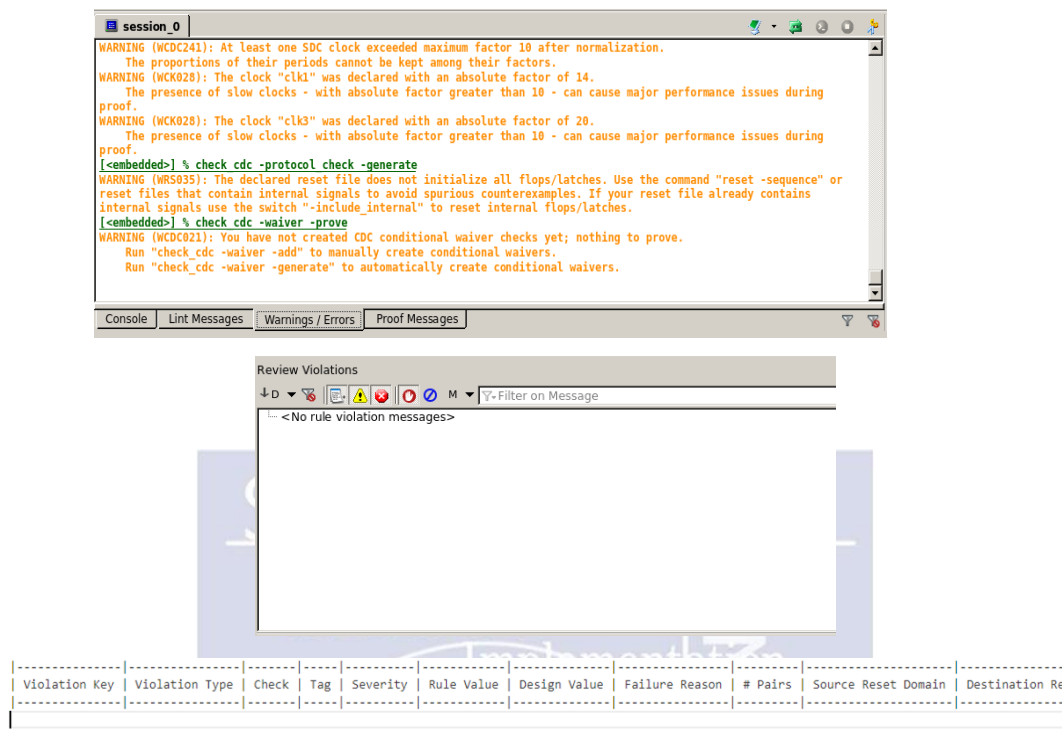


Fig7. Correct JG messages

Inputs

I/O	Signal name	Bits	Description
Input	clk1	1	Positive-edge trigger clock by clock 1 with clock period 14.1ns
Input	clk2	1	Positive-edge trigger clock by clock 2 with clock period 10.1ns
Input	clk3	1	Positive-edge trigger clock by clock 3 with clock period 20.7ns (01_RTL to 03_GATE) 4 different clk3 period (3.1ns, 4.1ns, 11.1ns, 20.7ns) in the 01_RTL stage
Input	rst_n	1	Asynchronous reset active low reset
Input	in_valid	1	in_data are valid when in_valid is high. This signal is triggered by clk1 .
Input	in_data	32	The coefficients of the input polynomial. This signal is triggered by clk1 for 16 cycles , each cycle contains 8 4-bit coefficients. One with lower degree is placed closer to LSB of in_data .

Outputs

I/O	Signal name	Bits	Description
output	out_valid	1	Should be set to low after reset and not be raised when in_valid is high. Should set to high when your out_data is ready. Should be asserted for 128 cycles, but is not required to be continuous.
output	out_data	16	The result of NTT. Should be output for 128 cycles, not required to be continuous.

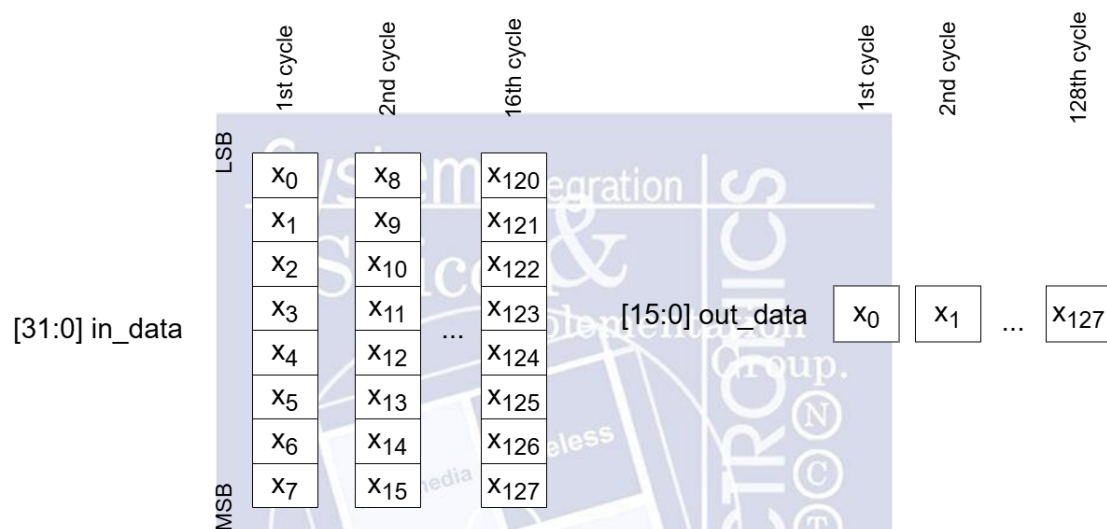


Fig8. Example of input_data and output_data in cycles.

Specifications

Modules

1. Top module name : NTT_TOP (File name: NTT_TOP.v)
2. Submodule name : CLK_1_MODULE, CLK_2_MODULE, CLK_3_MODULE (File name: DESIGN_MODULE.v)
3. Synchronizer name : Handshake_syn, FIFO_syn (File name: Handshake_syn.v, FIFO_syn.v in synchronizer folder)
4. Synchronizer from TA: NDFF_syn, NDFF_BUS_syn (File name: NDFF_syn.v, NDFF_syn.v in synchronizer folder)
5. Dual port SRAM : DUAL_64X16X1BM1 (04_MEM folder) (Words: 64, Bits: 16, Dual port SRAM)

Reset

6. Use **asynchronous** reset active low architecture.
7. The reset signal (rst_n) would be given only once at the beginning of simulation. All output signals should be reset after the reset signal is asserted.

Input/Output Signal

8. Input data are synchronous to clk1, output data are synchronous to clk3.
9. The out_data should be correct when out_valid is high.
10. The out_data should be reset after your out_valid is pulled down.
11. Output signal out_valid and out_data **should be zero when in_valid is high.**
12. The next input pattern will arrive 1~3 clk1 cycles after the 128th output data is received.
13. The output should be raised for 128 cycles and is not required to be continuous.

Synthesis and Prime Time

14. Output delay is $0.5 * \text{clk3 Clock Period}$.
15. Input delay is $0.5 * \text{clk1 Clock Period}$.
16. The output loading is set to 0.05.
17. **Your design area should not > 2000000.**
18. In the synchronizer you design, please use the NDFF_syn, NDFF_BUS_syn, provided by TA if needed. Prime Time will ONLY **set_annotated_check** to the NDFF_syn module provided by TA.
19. The NTT_TOP.sdc is complete by TA. **DO NOT modify it.** This file is to set the asynchronous clock groups of clk1, clk2, and clk3.

```
set_clock_groups -name group1 -asynchronous -group {clk1} -group {clk2} -group {clk3}
```

20. After synthesis, check the “NTT_TOP.area” and “NTT_TOP.timing” in the folder “Report”. The area report is valid only when the slack in the end of “NTT_TOP.timing” is **non-negative**.
21. The synthesis result cannot contain any **latch, error, violation, mismatch** (in syn.log).
22. After run Prime Time, the slack in the end of “NTT_TOP_pt.timing” should be also non-negative.
23. The Prime Time result cannot contain any **error, violation** (in syn.log).

Gate Level Simulation

24. **You can't have timing violation in gate-level simulation.**

Clock Period and Latency

25. The design should be able to operate at different output cycles. Please take advantage of the FIFO synchronizers. **TA will demo your design at 4 different clk3 period (3.1ns, 4.1ns, 11.1ns, 20.7ns) in the 01_RTL stage.**
26. **In the 02_SYN and 03_GATE stages, the clk3 period will be fixed at 20.7ns, and latency is calculated based on this.**
27. The latency is measured from the falling edge of in_valid to the falling edge of the final out_valid signal, inclusive of the output cycles!!!!!!
28. Your latency should be **smaller than 5000 cycles in clk3.**

Dos and Don'ts

29. Changing top module is prohibited.
30. Don't use Designware IP.
31. Calculate the result in CLK_2_MODULE, using other modules to perform the NTT algorithm is prohibited.
32. Changing clock period is prohibited. Use the clocks listed above.
33. TA had generated dual port SRAM for the FIFO synchronizer, and the files are stored in 04_MEM. **DO NOT modify them.**
34. **You should use SRAM provided by TA** to design your FIFO synchronizers to maintain the fairness of area performance.
35. **Don't modify the parameter "WIDTH" in Handshake_syn.v and FIFO_syn.v.**
36. Don't use any wire, reg, submodule, parameter names containing *error*, *congratulation*, *pass*, *latch* or *fail*. Doing so will result in failing the lab.
Note: * means any char in front of or behind the word, e.g: error_note is forbidden.
37. Don't write chinese comments or other language comments in the file you turned in.
Otherwise, **you will get 5 deduct points.**

Supplement

38. Some pre-defined flags are reserved for you to optimize your design.
39. It's acceptable for the following two warning.

```
Warning-[SDFCOM_CFTC] Cannot find timing check
NTT_TOP_SYN_pt.sdf, 63157
module: QDFFRBS, "instance: TESTBED.I NTT.u_Handshake_syn_dout_reg_28_"
SDF Warning: Cannot find timing check $hold(posedge CK,posedge RB,...)

** MEM_Warning: Read and Write the same Address, D0 is unknown ( 5138683050 ps) in clock of TESTBED.
I NTT.u FIFO_syn_u_dual_sram.ErrorMessage
```

Grading Policy

- ♦ Function correct 70% (01_RTL to 03_GATE, 4 different clk3 cycle times in 01_RTL)
- ♦ Jasper Gold correct 25%
- ♦ Performance: Latency * Area 5% (Latency is calculated in clk3)
If you didn't pass Function or JG, your score would not include performance.
- ♦ The grade of 2nd demo would be 30% off. And you won't get any points if you fail the function test.
Ex1: Pass function but fail JG in 1st demo. 70%
Ex2: Pass function in 1st demo and JG in 2nd demo. $70\% + (25\% + \text{performance } 5\%) * 0.7$
Ex3: Pass JG in 1st demo but fail function in 1st demo. 0%
Ex4: Pass JG in 1st demo and fail function in 2nd demo. $(70\% + 25\% + \text{performance } 5\%) * 0.7$
- ♦ **The latency is from the falling edge of in_valid to the falling edge of out_valid for the last output, INCLUDING the output cycles.**

Note

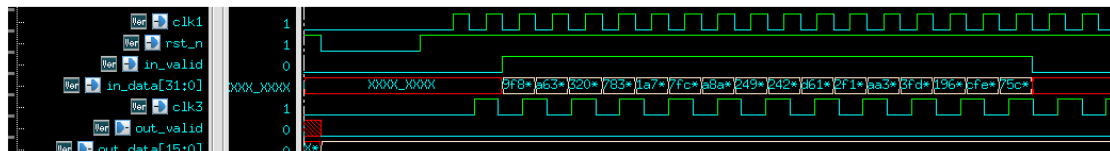
Template folders and reference commands:

1. 01_RTL/ (RTL simulation)
 - I. `./01_run_vcs_rtl`
2. 02_SYN/ (Synthesis)
 - I. `./01_run_dc_shell`
(Check the design which contains **latch and error** or not in **syn.log**)
 - II. `./02_run_pt`
(**set_annotated_check** for the first FF of NDFF synchronizer)
(Check the design's timing in /Report/ NTT_TOP_pt.timing)
3. 03_GATE_SIM/ (GL simulation)
 - I. `./01_run_vcs_gate`
(We will only run 20.7ns for clk3 period in GL simulation)
(Check no timing violation)
4. 05_JG/ (CDC verification)
 - I. `./01_run_jg`
5. 09_SUBMIT/ (submit file)
 - I. `./00_tar`
 - II. `./01_submit`
 - III. `./02_check`

1st demo deadline: 2025/11/03 (Mon.) 12:00:00
2nd demo deadline: 2025/11/05 (Wed.) 12:00:00
6. You can key in `./09_clean_up` to clear all log files and dump files in each folder.
7. You need to upload your design and system will name them as **DESIGN_module_iclabxx.v, Handshake_syn_iclabxx.v, FIFO_syn_iclabxx.v.** (you should check with `./02_check {1st_demo/2nd_demo}`)
8. If the uploaded file violating the naming rule, you will get 5 deduct points.

Waveform Example

1. Asynchronous, active-low reset that resets all outputs.
2. 16 cycles for input signals. Each cycle contains eight 4-bit coefficients from the input polynomial.



3. 128 cycles for output signals and not required to be continuous.

